

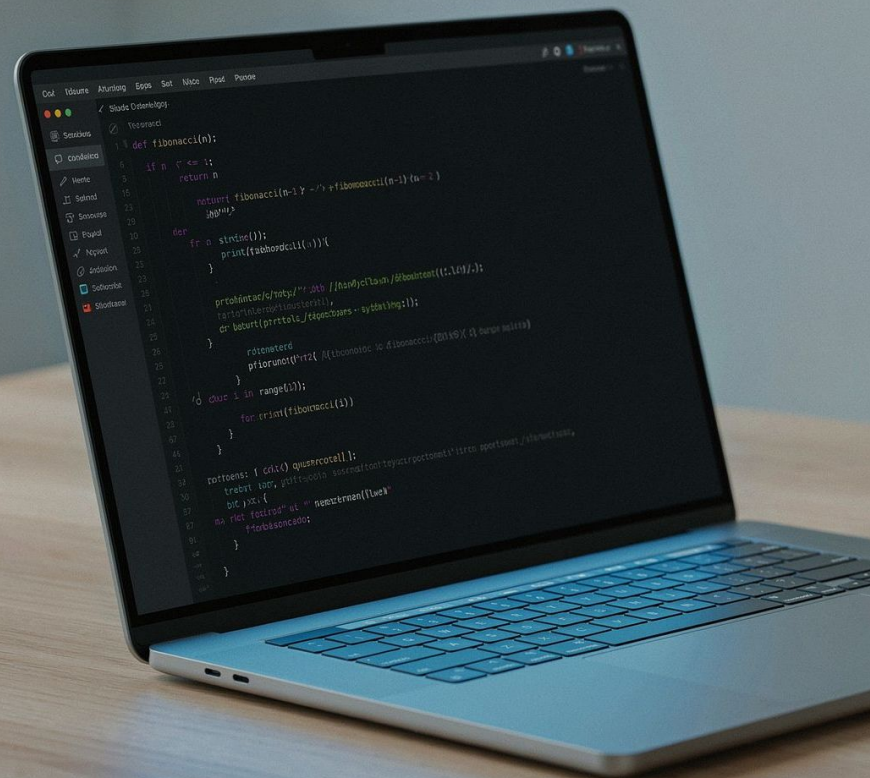
Streamlit — od skryptu do aplikacji

Praktyczny przewodnik po budowaniu interaktywnych aplikacji w Pythonie: od pierwszego uruchomienia, przez architekturę i widgety, aż po produkcyjny deployment i studia przypadków.

PYTHON

DATA SCIENCE

ML APPS



Dlaczego Streamlit?

Problem

Tradycyjne podejście do budowy aplikacji webowej wymaga znajomości HTML, CSS, JavaScript, routingu i szablonów — co dla inżynierów danych to niepotrzebne rozciąganie cyklu pracy.

Rozwiązanie

Streamlit redukuje tę barierę do minimum. **Pisz aplikację tak, jak piszesz skrypt w Pythonie.** Wywołujesz funkcje, Streamlit tłumaczy je na elementy interfejsu i renderuje w przeglądarce.

- Dashboardy analityczne i eksploracja danych
- Interfejsy do testowania modeli ML
- Narzędzia wewnętrzne i prototypy
- Proste integracje z API i usługami chmurowymi

❏ Streamlit nie zastępuje klasycznych frameworków webowych przy dużych, wieloużytkownikowych serwisach produkcyjnych — to narzędzie do szybkiej iteracji i prezentacji.



Model wykonania Streamlit

Kluczowa zasada: Streamlit wykonuje cały skrypt od początku przy każdej interakcji użytkownika.

1

Odczyt stanu

st.session_state przechowuje wartości między przebiegami

2

Wykonanie skryptu

Skrypt uruchamiany od pierwszej linii

3

Renderowanie

Wynik trafia do przeglądarki użytkownika

Myśl o Streamlit jak o interaktywnym notebooku wykonywanym od początku przy każdej zmianie parametrów. **session_state** to mechanizm przechowywania wartości między kolejnymi przebiegami skryptu.

Streamlit vs inne narzędzia

Narzędzie	Zastosowanie	Zalety	Wady
Streamlit	Prototypy, dashboardy, ML	Szybkie prototypowanie, integracja z Pythonem	Ograniczona kontrola nad frontendem
Flask	API, backend	Pełna kontrola nad routingiem	Wymaga budowy frontendu
Django	Duże aplikacje	Kompletne rozwiązanie (ORM...)	Większa złożoność
Dash	Analityka, wykresy	Integracja z Plotly	Bardziej rozbudowany model komponentów
Gradio	Interfejsy ML	Szybkie UI dla modeli	Mniejsza elastyczność

Wybór narzędzia zależy od wymagań projektu: szybkość prototypowania vs. potrzeba kontroli i skalowalności.

Instalacja i środowisko

Wymagania wstępne

- Python 3.8+ (zalecane 3.10 lub 3.11)
- pip lub conda do zarządzania pakietami
- Przeglądarka: Chrome, Firefox, Edge
- System: Linux, macOS lub Windows

Tworzenie środowiska (venv)

```
python -m venv .venv
source .venv/bin/activate # macOS/Linux
.venv\Scripts\Activate.ps1 # Windows
pip install streamlit
```

Pierwsza aplikacja

```
import streamlit as st
st.set_page_config(
    page_title="Przykład",
    layout="wide")
st.title("Hello, Streamlit!")
st.write("Moja pierwsza aplikacja.")
if st.button("Kliknij mnie"):
    st.write("Dziękuję!")
```

Uruchomienie

```
streamlit run app.py
# → http://localhost:8501
```

📌 Zawsze pracuj w izolowanym środowisku wirtualnym i zarządzaj zależnościami przez requirements.txt lub poetry.lock — ułatwia to reprodukcję środowiska.



Struktura projektu i modularność

Warstwa prezentacji — `app.py`

Cienka warstwa wywołująca funkcje z `src/` i renderująca wynik. Traktuj ją jak kontroler, nie jak miejsce na logikę.

Logika biznesowa — `src/`

Funkcje ładujące dane (`data.py`), model (`model.py`) i integracje (`services.py`) — testowalne niezależnie od Streamlit.

Podstrony — `pages/`

Folder `pages/` pozwala rozdzielić większe sekcje aplikacji na osobne pliki, automatycznie tworząc nawigację.

Reprodukowalność — Docker

Dockerfile + requirements.txt lub poetry.lock gwarantują identyczne środowisko lokalnie, w CI i w produkcji.

Layout i komponenty — zasady projektowe

1 Separacja sterowania od treści

Filtry i parametry w sidebar lub wyraźnie oddzielonej kolumnie. Treść główna skupiona na wynikach.

2 Hierarchia informacji

Nagłówki, podtytuły, krótkie opisy. Użytkownik powinien od razu wiedzieć, co jest najważniejsze.

3 Minimalizm interakcji

Każdy widget to potencjalne źródło nieporozumień — dodawaj tylko te, które naprawdę wpływają na wynik.

4 Informowanie o stanie

Długie operacje, ładowanie, błędy — komunikuj użytkownikowi przez spinner, progress i komunikaty.

Kluczowe kontenery

- **st.sidebar** — filtry i ustawienia globalne
- **st.columns** — układy poziome z proporcjami
- **st.container** — grupowanie logiczne elementów
- **st.expander** — ukrywanie szczegółów i logów
- **st.tabs** — zakładki dla powiązanych widoków

Interaktywne widgety i wzorce

Natychmiastowa reakcja

Dla lekkich operacji: filtrowanie tabeli, zmiana zakresu wykresu, podgląd parametrów. Streamlit reaguje automatycznie.

Wzorzec „Zastosuj”

Dla kosztownych operacji (trening modelu, zapytania API) użyj przycisku `st.button("Zastosuj")` oddzielającego konfigurację od wykonania.

Formularze (st.form)

Zbierz wiele pól i wyślij jednorazowo. Idealne gdy chcesz walidować wszystkie pola razem i wykonać jedną akcję.

Dla każdego widgetu zawsze podawaj **jasną etykietę**, **wartość domyślną** i **walidację** — użytkownik powinien wiedzieć, co zmienia i dlaczego.



Najczęstsze błędy w Streamlit

1

Traktowanie jak SPA

Streamlit wykonuje cały skrypt od nowa — używaj `session_state` zamiast zmiennych lokalnych.

2

Brak cache'owania

Kosztowne operacje bez `@st.cache_data` spowalniają aplikację przy każdej interakcji.

3

Ciężkie obiekty w `session_state`

Przechowuj identyfikatory zasobów, nie same obiekty. Dla modeli i połączeń używaj `@st.cache_resource`.

4

Monolit `app.py`

Mieszanie logiki danych, modelu i prezentacji utrudnia testowanie. Dziel na moduły: `pages/`, `src/`.

5

Sekrety w repozytorium

Nigdy nie commituj kluczy API. Używaj `st.secrets` lub zmiennych środowiskowych.

Zarządzanie stanem — session_state w praktyce

Zasady projektowe stanu

- **Jedno źródło prawdy** — unikaj rozproszonego stanu w zmiennych globalnych
- **Minimalny stan w UI** — przechowuj tylko to, co musi przetrwać między przebiegami
- **Inicjalizacja przed użyciem** — zawsze inicjalizuj klucze, by uniknąć KeyError
- **Atomowe aktualizacje** — przypisuj nowy obiekt zamiast modyfikować zagnieżdżone struktury

Wzorce zarządzania stanem

- **Session Store** — prosty stan (liczniki, filtry) bezpośrednio w session_state
- **Client State + Server Resource** — identyfikatory w session_state, ciężkie zasoby w @st.cache_resource
- **Krokowy wizard** — session_state przechowuje krok i tymczasowe dane formularza

⚠ Brak migracji stanu po aktualizacji aplikacji może łamać sesje użytkowników. Wersjonuj klucze stanu i wykonuj migracje przy starcie.



Praca z danymi i wydajność



Małe i średnie pliki

Używaj `@st.cache_data` dla funkcji ładujących CSV lub wyniki zapytań SQL. Cache jest kluczem do responsywności.



Duże pliki — chunking

Dla plików wielogigabajtowych używaj `chunksize` w Pandas lub strumieniowego odczytu. Agreguj wyniki pośrednie, nie ładuj wszystkiego naraz.



Zewnętrzne źródła

Wykonuj agregacje po stronie serwera SQL. Pobieraj tylko potrzebne kolumny i wiersze. Paginuj wyniki API.



Profilowanie

Mierz czas (`time.perf_counter`) i pamięć (`df.memory_usage`) zanim zaczniesz optymalizować. Największe zyski dają zmiany architektoniczne.

Wizualizacje — wybór biblioteki

Biblioteka	Interaktywność	Typowe zastosowania	Kiedy wybrać
Plotly	Wysoka	Interaktywne dashboardy, eksploracja	Domyślny wybór dla UI z widgetami
Altair	Wysoka	Wykresy statystyczne, deklaratywne	Szybkie, czytelne wykresy z Pandas
Matplotlib	Niska	Publikacyjne, niestandardowe kompozycje	Pełna kontrola nad wyglądem
Bokeh	Wysoka	Zaawansowane callbacki klienta	Custom interakcje po stronie klienta

Agreguj dane przed rysowaniem — renderowanie milionów punktów w przeglądarce jest kosztowne. Dla bardzo dużych zbiorów używaj **Plotly Scattergl** (WebGL) lub **DataShader** w Bokeh. Zawsze ustawiaj `use_container_width=True` dla responsywności.

Modele ML w Streamlit

Ładowanie modeli

Ładuj model **raz na proces** i współdziel między wywołaniami. Nigdy nie trzymaj modeli w session_state.

```
@st.cache_resource
def load_model(path):
    import joblib
    return joblib.load(path)

model = load_model("models/model.pkl")
```

Tryby inferencji

- **Synchroniczny** — niskie opóźnienia, pojedyncze zapytania
- **Batchowy** — lepsza przepustowość GPU/TPU
- **Asynchroniczny** — delegowanie do workerów, powiadomienie po zakończeniu

Formaty modeli

ONNX

Interoperacyjność między frameworkami

TorchScript

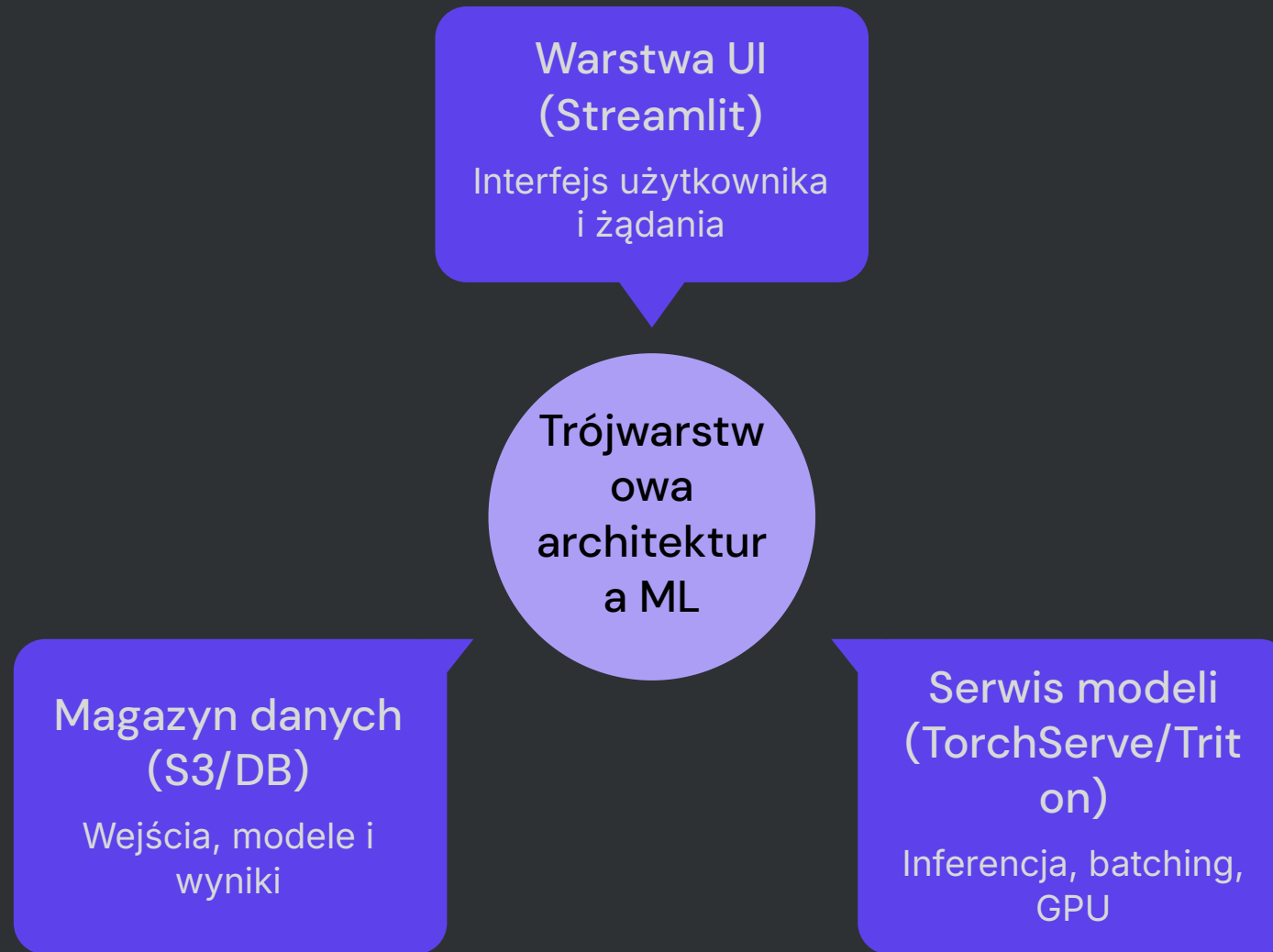
PyTorch produkcja, C++/Python

Joblib

Szybkie prototypy scikit-learn

⚠ Nie ładuj pickli z nieznanych źródeł — deserializacja może wykonać złośliwy kod.

Architektura rekomendowana dla ML



Dla skalowalnych aplikacji ML oddziel warstwę UI (Streamlit) od warstwy inferencji (dedykowany serwis modelowy przez HTTP/gRPC). Streamlit deleguje zapytania, serwis obsługuje batching, GPU i skalowanie. Dodaj wyjaśnialność (SHAP, LIME) i monitorowanie driftu danych.

Deployment — opcje i strategie



Streamlit Cloud

Najprostsze, szybkie demo. Ograniczone zasoby i brak pełnej kontroli.



PaaS (Heroku, Render)

Małe aplikacje produkcyjne. Proste CI i automatyczne deploje.



Docker + VPS

Pełna kontrola. Ustaw `--server.address=0.0.0.0` w kontenerze.



Kubernetes

Skalowanie i orkiestracja dla aplikacji wymagających wysokiej dostępności.

CI/CD, monitoring i bezpieczeństwo

Pipeline CI/CD

01

Build obrazu Docker

02

Testy jednostkowe i linting

03

Skan bezpieczeństwa zależności

04

Deploy + smoke tests

05

Promocja do produkcji

Monitoring i bezpieczeństwo

- Strukturalne logi JSON — centralizuj w ELK, Loki lub Datadog
- Mierz latencję (p50, p95, p99) i zużycie pamięci
- Health checks: liveness i readiness probes
- Sekrety w menedżerze (Vault, AWS Secrets Manager) — nigdy w repozytorium
- Szyfruj ruch TLS/HTTPS, ogranicz dostęp sieciowy
- Strategie wdrożeń: **blue-green** lub **canary** dla krytycznych zmian

Case Study: Dashboard analityczny

Architektura

- UI: Streamlit (app.py)
- Dane: PostgreSQL z agregacjami po stronie serwera
- Cache: @st.cache_data dla zapytań, @st.cache_resource dla połączenia DB
- CI/CD: Docker + GitHub Actions

Kluczowe zasady

- Agreguj w SQL, nie w Pandas
- Paginuj szczegółowe tabele
- Indeksy na kolumnach daty i regionu
- Health check DB i retry logic

Wzorzec kodu

```
# src/queries.py
@st.cache_data
def load_aggregates(conn, start, end, region):
    return pd.read_sql(query, conn,
                       params=(start, end, region))

# app.py
conn = get_conn(st.secrets["DB_CONN"])
start, end = st.sidebar.date_input(
    "Zakres dat", [default_start, default_end])
region = st.sidebar.selectbox(
    "Region", ["EU", "US", "APAC"])
df = load_aggregates(conn, start, end, region)
st.plotly_chart(
    px.line(df, x="day", y="sales"),
    use_container_width=True)
```

Case Study: Aplikacja inferencyjna ML i duże dane



Aplikacja inferencyjna

Upload CSV → preprocessing → predykcja → wyjaśnienia SHAP. Dla długich zadań: kolejka Redis + Celery, wyniki w S3/DB, UI polluje status. Batchuj zapytania do GPU, ogranicz rozmiar wejścia.



Eksploracja dużych danych

Dane w Parquet na S3 lub ClickHouse/BigQuery. Pre-agregacje i materializowane widoki dla typowych zapytań. Streamlit jako UI z paginacją i lazy loading. Transformacje w Spark lub Dask.



Przetwarzanie asynchroniczne

Streamlit UI enqueue'uje zadania do Redis/Celery. Worker wykonuje analizę i zapisuje wynik. UI polluje endpoint statusu. Ustaw limity concurrency, retry i dead letter queue.

Checklist przed produkcją

Kod i testy

- ✓ Unit testy dla logiki w src/
- ✓ Linter i formatowanie kodu
- ✓ Type checks
- ✓ Testy integracyjne kluczowych ścieżek

Bezpieczeństwo

- ✓ Sekrety w menedżerze, nie w repozytorium
- ✓ TLS/HTTPS i ograniczenia sieciowe
- ✓ Skan zależności pod kątem luk
- ✓ Walidacja i sanityzacja wejść użytkownika

Operacje

- ✓ Docker image z tagowaniem semantycznym
- ✓ CI/CD pipeline z testami i skanami
- ✓ Health checks i readiness probes
- ✓ Centralne logowanie i metryki z alertami

Rollback i DR

- ✓ Tagowane obrazy do rollbacku
- ✓ Migracje bazy odwracalne
- ✓ Playbook awaryjny z procedurami
- ✓ Plan strategii wdrożenia (canary/blue-green)

Podsumowanie

Model wykonania

Skrypt od nowa przy każdej interakcji. `session_state` i `cache` to fundamenty przewidywalnej aplikacji.

Architektura

Oddziel prezentację (`app.py`) od logiki (`src/`). Testuj funkcje niezależnie od Streamlit.

Wydajność

Buforuj kosztowne operacje, agreguj po stronie serwera, przetwarzaj w partiach, profiluj przed optymalizacją.

Produkcja

Docker, CI/CD, monitoring, bezpieczne sekrety i plan rollbacku to podstawa stabilnego serwisu.

Streamlit skraca drogę od pomysłu do działającej aplikacji — pod warunkiem, że rozumiesz jego model wykonania i stosujesz dobre praktyki projektowe.

